Industry-Ready Advanced SQL Tutorial

Realistic Use Cases from Netflix, Amazon, Uber, Tesla, Microsoft

Dr. Sumaiya Tabassum Nimi
Assistant Professor, Dept. of ECE — North South University

Introduction

Modern DBMS jobs at companies such as **Amazon, Microsoft, Netflix, Uber, Tesla, Airbnb** require strong SQL skills beyond basic SELECT. This handout focuses on industry-relevant concepts:

- Complex **JOINs**
- Nested **subqueries**
- **CTEs** (Common Table Expressions)
- Window functions (industry standard)
- Aggregation with filtering
- Real corporate-style data analysis tasks

Dataset Overview

We assume the following business-style tables, similar to what engineers use in Netflix, Amazon, and Uber:

- **Users(user_id, name, country)**
- **Subscriptions(user_id, plan, monthly_fee, start_date)**
- **WatchHistory(user_id, movie_id, watch_time)**
- **Movies(movie_id, title, category, rating)**
- **Rides(ride_id, driver_id, rider_id, city, fare, timestamp)**
- **Drivers(driver_id, name, rating)**

Case Study 1 — Netflix: "Find Your Power Viewers"

Problem

Netflix wants to find the **top 5 users** (per country) who have watched the **most total minutes** of movies.

SQL Solution

```
WITH TotalWatch AS (
   SELECT
      u.user_id,
      u.name,
      u.country,
      SUM(w.watch_time) AS total_minutes
   FROM Users u
   JOIN WatchHistory w ON u.user_id = w.user_id
   GROUP BY u.user_id, u.name, u.country
),
Ranked AS (
   SELECT *,
          DENSE_RANK() OVER (PARTITION BY country ORDER BY total_minutes DESC) AS rk
   FROM TotalWatch
)
SELECT *
FROM Ranked
WHERE rk <= 5;
```

Industry insight: Netflix uses window functions extensively to rank users across geographies. This query mirrors their real viewership segmentation pipelines.

Case Study 2 — Amazon Prime: "Identify Revenue-Leading Plans"

Problem

Amazon wants to find which subscription plan generates the **highest revenue** overall.

SQL Solution

```
SELECT plan, SUM(monthly_fee) AS total_revenue
FROM Subscriptions
GROUP BY plan
ORDER BY total_revenue DESC
LIMIT 1;
```

Why this matters: Such queries help Amazon track ARPU (Average Revenue per User).

Case Study 3 — Uber: "Detect Cities With Highest Rider Activity"

Problem

Uber wants to find cities where:

- the number of rides exceeds 1000 per day
- AND the average fare is more than $12

SQL Solution

```
SELECT city,
       COUNT(*) AS total_rides,
       AVG(fare) AS avg_fare
FROM Rides
GROUP BY city
HAVING COUNT(*) > 1000
   AND AVG(fare) > 12;
```

Industry relevance: Uber operations teams use EXACTLY this style of HAVING+GROUP BY filtering.

Case Study 4 — Tesla: "Driver Performance Analytics"

Problem

Tesla wants to know:

Which drivers have ratings ABOVE the average rating across all Tesla drivers?

SQL Solution

```
SELECT d.driver_id, d.name, d.rating
FROM Drivers d
WHERE d.rating > (
   SELECT AVG(rating)
   FROM Drivers
);
```

Subquery pattern: Very common industry technique for selecting top performers.

Case Study 5 — Microsoft Azure: "Detect Underutilized Subscriptions"

Problem

Microsoft wants to find users who:

- pay for the premium plan - but have watched LESS than the average watch-time of all users

SQL Solution

```
SELECT s.user_id
FROM Subscriptions s
JOIN WatchHistory w ON s.user_id = w.user_id
WHERE s.plan = 'Premium'
GROUP BY s.user_id
HAVING SUM(w.watch_time) < (
   SELECT AVG(total_watch)
   FROM (
      SELECT user_id, SUM(watch_time) AS total_watch
      FROM WatchHistory
      GROUP BY user_id
   ) X
);
```

Industry relevance: Microsoft uses similar SQL for subscription churn prediction.

Case Study 6 — Amazon: "Customers Who Watch the Same Movies"

Problem

Find all pairs of users who watched at least 10 of the same movies.

SQL Solution

This is a classic Amazon "similar user" query.

```
SELECT
    w1.user_id AS userA,
    w2.user_id AS userB,
    COUNT(*) AS common_movies
FROM WatchHistory w1
JOIN WatchHistory w2
    ON w1.movie_id = w2.movie_id
   AND w1.user_id < w2.user_id
GROUP BY w1.user_id, w2.user_id
HAVING COUNT(*) >= 10;
```

Pattern used in Amazon personalization engines.

Case Study 7 — Netflix: "Movie Category Popularity"

Problem

Find the **average rating** per movie category, sorted from most to least liked.

SQL Solution

```
SELECT category, AVG(rating) AS avg_rating
FROM Movies
GROUP BY category
ORDER BY avg_rating DESC;
```

Case Study 8 — Uber: "Identify Top Earning Drivers in each City"

Problem

For each city, find the **top 3 drivers** with highest total fare.

SQL Solution

```
WITH DriverEarnings AS (
   SELECT d.driver_id, d.name, r.city,
          SUM(r.fare) AS total_earnings
   FROM Drivers d
   JOIN Rides r ON d.driver_id = r.driver_id
   GROUP BY d.driver_id, d.name, r.city
),
Ranked AS (
   SELECT *,
      ROW_NUMBER() OVER (PARTITION BY city ORDER BY total_earnings DESC) AS rn
   FROM DriverEarnings
)
SELECT *
FROM Ranked
WHERE rn <= 3;
```

Conclusion

These realistic SQL problems mirror real tasks completed by engineers at:

- Netflix — content recommendation, viewership analytics
- Amazon — customer segmentation, revenue modeling
- Uber — ride forecasting, driver performance
- Tesla — sensor analytics, fleet telemetry
- Microsoft — cloud subscription analytics

Mastering subqueries, joins, window functions, and CTEs will prepare you for real DBMS engineering roles.